

Photoshop da Julia: Da Teoria à Prática em Processamento de Imagens

Julia S. R. Gomes

Departamento de Ciência da Computação

Univ. Federal Rural do Rio de Janeiro

Nova Iguaçu, RJ, Brasil

Email: julia.robaina8@gmail.com

Resumo—Neste trabalho foi implementado um programa editor de imagens para por em prática conhecimentos teóricos da área de processamento de imagens. No editor de imagens, as funcionalidades de carregar, aplicar um filtro, operação de segmentação, operação de morfologia e salvar a imagem estão disponíveis. Para testar a eficácia da aplicação, imagens com e sem ruído foram manipuladas. O programa se mostrou apto a executar as funções que foram propostas.

I. INTRODUÇÃO

Processamento de Imagens é a área de estudo responsável por realizar tratamentos em uma imagem para atingir um objetivo. Ou seja, "é a manipulação de uma imagem por computador de modo onde a entrada e a saída do processo são imagens" [1]. Neste artigo, será abordado técnicas de filtragem no domínio do espaço lineares e não-lineares: filtro média, mediana, gaussiano, sobel e laplaciano; segmentação: binarização, limiarização global e adaptativa; morfologia: erosão, dilatação, abertura e fechamento. Todos esses filtros e operações foram condensados em um programa editor de imagens, chamado Photoshop da Julia com o objetivo de por em prática conhecimentos teóricos da área de processamento de imagens. Quanto à organização do trabalho, na seção II encontra-se a conceituação das técnicas utilizadas, na seção III há a descrição e disponibilização do código desenvolvido para a criação do editor, na seção IV os resultados obtidos por meio de observações de imagens manipuladas pelo programa, na seção V está a conclusão e na seção VI está as imagens da seção de resultados, seguida das referências.

II. TÉCNICAS UTILIZADAS

A. Filtros Lineares

Utilizam a operação de convolução entre a matriz representante da imagem com a matriz de convolução representante do tipo de filtro, a matriz de convolução também pode ser chamada de máscara ou kernel. A operação de convolução funciona de tal forma que cada pixel da matriz imagem sofrerá uma combinação linear com uma máscara de convolução e o pixel central da máscara deverá ser multiplicado com o pixel central da matriz imagem. Isso produzirá um novo valor de pixel para criar uma imagem com o efeito do filtro escolhido. Para a realização do cálculo, a matriz imagem precisa ser adicionada linhas e colunas com valores zero ou a cópia de seus valores das linhas e colunas das extremidades para que

a operação de convolução possa ser realizada corretamente no programa. Nesta aplicação, tanto a adição com valor zero quanto a adição de cópia dos valores extremos foram utilizadas. Na figura 1, um exemplo da operação de convolução é exibido. Os filtros lineares implementados no programa foram, filtro gaussiano, sobel e laplaciano.

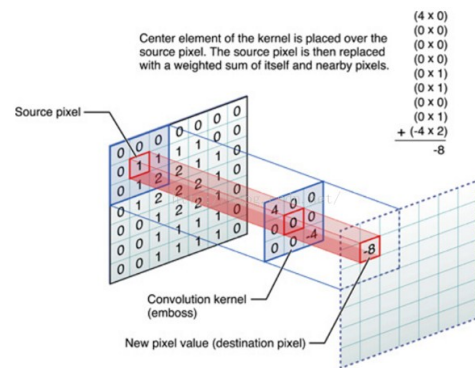


Fig. 1. Convolução Fonte: [2]

B. Filtros Não-Lineares

São filtros que não usam a operação de convolução ou transformadas de Fourier [3]. Os filtros não-lineares implementados no programa foram filtro média e mediana.

C. Filtros Passa-Baixa

Os filtros passa-baixa permitem que baixas frequências passem enquanto as altas frequências são atenuadas, portanto após o uso do filtro, os valores de cada pixel da matriz resultante tendem a ser menores do que a matriz original. Também são conhecidos como filtros de suavização, pois removem ou suavizam ruídos, bordas e transições de intensidade bruscas em uma imagem [4]. Um cuidado com esses filtros é que ao usá-los pode acontecer perda de detalhes importantes para a imagem, causando um borramento excessivo [5]. Adiante é apresentado os filtros de suavização utilizados neste trabalho.

1) *Filtro Média*: cada novo pixel recebe o resultado de uma combinação linear dividido pelo somatório dos valores do kernel. Esse somatório de valores do kernel também é conhecido como fator de normalização [5]. O filtro média

segue o raciocínio da média aritmética ponderada. Nas figuras 2 e 3 é mostrado os kernels de filtro média usados.

$$Z = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad Z = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (1)$$

Fig. 2. Kernel 3x3 e kernel 5x5: [6]

$$Z = \frac{1}{49} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (2)$$

Fig. 3. kernel 7x7 Fonte: [6]

2) *Filtro Mediana*: de acordo com um tamanho de kernel, o seu pixel central ficará sobre o pixel alvo da matriz imagem e todos os valores que estiverem incluídos na matriz imagem de acordo com o tamanho do kernel serão ordenados e o valor da mediana será escolhido. O cálculo segue a mesma linha de raciocínio do cálculo da mediana em estatística e pode ser visualizado na figura 4.

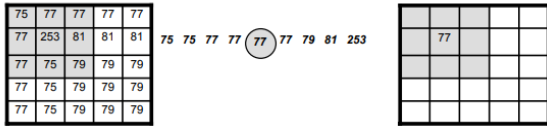


Fig. 4. Mediana 3x3 Fonte: [7]

3) *Filtro Gaussiano*: os mesmos passos para o cálculo do novo pixel de uma imagem com filtro média é feito para o filtro gaussiano. A primeira diferença é que, na matriz de convolução gaussiana, o pixel central é o pixel de maior valor e os valores dos demais pixels vão decrescendo do centro para as extremidades. A linha de raciocínio aqui empregado é o mesmo de uma função gaussiana. A segunda diferença é que esse filtro sofre influência de um desvio padrão, como pode ser visto na fórmula da figura 5; quanto maior for o desvio padrão, maior será o nível de suavização. As máscaras utilizadas neste trabalho estão com desvio padrão igual a 1 e podem ser visualizadas nas figuras 6 e 7.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}} \quad (3)$$

Fig. 5. Fórmula Gaussiana em 2D Fonte: [6]

As diferenças entre esses três filtros são que, o filtro média é incapaz de preservar bordas e detalhes da imagem ao contrário

$$Z = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad Z = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix} \quad (4)$$

Fig. 6. Kernel 3x3 e Kernel 5x5 Fonte: [6]

$$Z = \frac{1}{4096} \begin{bmatrix} 1 & 6 & 15 & 20 & 15 & 6 & 1 \\ 6 & 36 & 90 & 120 & 90 & 36 & 6 \\ 15 & 90 & 225 & 300 & 225 & 90 & 15 \\ 20 & 120 & 300 & 400 & 300 & 120 & 20 \\ 15 & 90 & 225 & 300 & 225 & 90 & 15 \\ 6 & 36 & 90 & 120 & 90 & 36 & 6 \\ 1 & 6 & 15 & 20 & 15 & 6 & 1 \end{bmatrix} \quad (5)$$

Fig. 7. Kernel 7x7 Fonte: [4]

do filtro mediana e gaussiano onde há uma degradação menor [8]. Em relação aos ruídos, filtros mediana são bons em remover ruídos sal e pimenta em comparação aos filtros de média e para ruídos gaussianos o filtro mediana tem um nível satisfatório em relação ao filtro média [8]. De acordo com [5] e [9] o filtro gaussiano, por seguir o cálculo da função gaussiana, ele faz uma suavização gradual e circunferencialmente simétrica. Além disso, o filtro gaussiano também é o filtro mais próximo da visão humana [9].

D. Filtros Passa-Alta

Os filtros passa-alta permitem que altas frequências passem enquanto as baixas frequências são atenuadas, portanto após o uso do filtro os valores de cada pixel da matriz resultante tendem a ser maiores do que a matriz original. Também são conhecidos como filtros de aguçamento, pois realçam bordas, transições de intensidade bruscas e detalhes em uma imagem [4].

1) *Filtro Sobel*: calcula o gradiente nas orientação vertical (Gy) e/ou horizontal (Gx) para identificar áreas que tenham altas diferenças de intensidade, como bordas. Isso é feito por meio do operador de sobel com as máscaras de convolução representante dos gradientes. As máscaras são convoluídas com a matriz imagem; após isso, o gradiente total é calculado. Nesta aplicação, o cálculo é feito pela soma entre os valores das duas matrizes, como mostrado na figura 8. O filtro sobel deste trabalho utiliza um filtro gaussiano 3x3 antes da operação de sobel para que ruídos que possam prejudicar o resultado da operação sejam removidos. Nas figuras 9, 10 e 11 é mostrado os kernels do filtro sobel usados.

$$G = |G_x| + |G_y| \quad (6)$$

Fig. 8. Gradiente Total Fonte: [10]

$$G_x = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (7)$$

Fig. 9. Kernel 3x3 Fonte: [11]

$$G_x = \begin{bmatrix} -1 & -4 & -6 & -4 & -1 \\ -2 & -8 & -12 & -8 & -2 \\ 0 & 0 & 0 & 0 & 0 \\ 2 & 8 & 12 & 8 & 2 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix} \quad (8)$$

$$G_y = \begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -4 & -8 & 0 & 8 & 4 \\ -6 & -12 & 0 & 12 & 6 \\ -4 & -8 & 0 & 8 & 4 \\ -1 & -2 & 0 & 2 & 1 \end{bmatrix} \quad (9)$$

Fig. 10. Kernel 5x5 Fonte: [12]

2) *Filtro Laplaciano*: de acordo com [4], "o filtro laplaciano corresponde à derivada de segunda ordem para a função $f(x, y)$ ". Seu cálculo está presente na figura 12. As matrizes de convolução desse filtro podem estar com o valor central positivo e a partir dele para todas as extremidades os valores são negativos, ou, o valor central negativo e a partir dele para todas as extremidades os valores são positivos [4]. O filtro laplaciano deste trabalho utiliza um filtro gaussiano 3x3 antes da operação de convolução para que ruídos que possam prejudicar o resultado da operação sejam removidos. É possível ver os filtros laplaciano usado neste trabalho nas figuras 13, 14 e 15.

A diferença entre o filtro sobel e o laplaciano é que o

$$G_x = \begin{bmatrix} -1 & -4 & -6 & -4 & -1 & 0 & 1 \\ -2 & -8 & -12 & -8 & -2 & 0 & 2 \\ -3 & -12 & -18 & -12 & -3 & 0 & 3 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 12 & 18 & 12 & 3 & 0 & -3 \\ 2 & 8 & 12 & 8 & 2 & 0 & -2 \\ 1 & 4 & 6 & 4 & 1 & 0 & -1 \end{bmatrix} \quad (10)$$

$$G_y = \begin{bmatrix} -1 & -2 & -3 & 0 & 3 & 2 & 1 \\ -4 & -8 & -12 & 0 & 12 & 8 & 4 \\ -6 & -12 & -18 & 0 & 18 & 12 & 6 \\ -4 & -8 & -12 & 0 & 12 & 8 & 4 \\ -1 & -2 & -3 & 0 & 3 & 2 & 1 \end{bmatrix} \quad (11)$$

Fig. 11. Kernel 7x7 Fonte: [12]

$$\nabla^2 f(x, y) = \frac{\partial^2}{\partial x^2} f(x, y) + \frac{\partial^2}{\partial y^2} f(x, y) \quad (12)$$

Fig. 12. Derivada de segunda ordem para uma função 2D Fonte: [4]

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \quad (13)$$

Fig. 13. kernel 3x3 Fonte: [12]

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & -1 & -2 & -1 & 1 \\ 1 & -2 & -4 & -2 & 1 \\ 1 & -1 & -2 & -1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix} \quad (14)$$

Fig. 14. Kernel 5x5 Fonte: [12]

$$\begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 2 & 2 & 2 & 1 & 0 \\ 1 & 2 & 4 & 8 & 4 & 2 & 1 \\ 1 & 2 & 8 & -24 & 8 & 2 & 1 \\ 1 & 2 & 4 & 8 & 4 & 2 & 1 \\ 0 & 1 & 2 & 2 & 2 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix} \quad (15)$$

Fig. 15. Kernel 7x7 Fonte: [12]

filtro de sobel detecta bordas na direção horizontal e vertical, enquanto o laplaciano pode identificar bordas independente da direção [13].

E. Segmentação

De acordo com [3], "a segmentação divide uma imagem em regiões ou objetos que a compõem". Isso é realizado para destacar objetos de interesse na imagem [3]. Na aplicação foi implementada a segmentação por limiarização, também chamada de *thresholding*, com a binarização da imagem original. A binarização consiste em converter a imagem em uma imagem binária, onde cada pixel pode assumir uma entre duas cores desejadas, geralmente representados por preto e branco. Essas cores são usadas para representar os objetos de interesse e o fundo. A limiarização define um valor numérico (limiar) que vai ser utilizado para binarizar a imagem. A limiarização pode ser dividida em dois tipos:

- Limiarização Global: somente um valor de limiar vai ser utilizado para binarizar todos os pixels da imagem. As técnicas desse tipo implementadas são a limiarização global simples e a limiarização pelo método de Otsu;
- Limiarização Adaptativa: a imagem é dividida em janelas de tamanho $n \times n$ e para cada janela um valor de limiar é calculado e utilizado para a binarização daquela parte da imagem. As técnicas desse tipo implementadas são a limiarização adaptativa pelo método de Otsu, a limiarização adaptativa por média e a limiarização adaptativa por Bernsen.

Em seguida, cada técnica mencionada receberá mais detalhes.

1) *Limiarização Global Simples*: criado por Ridler e Calward, essa técnica utiliza a média entre dois grupos distintos, que tem os pixels pertencentes a cada grupo determinado por

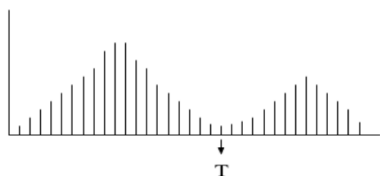


Fig. 16. Histograma Bimodal Fonte: [15]

um limiar [14]. Esse valor de limiar é alterado até que ele seja menor que um valor de limiar pré-determinado. limiarização global simples é boa para ser utilizada quando há uma grande diferença de intensidades na imagem, iluminação uniforme ou um *background* homogêneo e quando o histograma da imagem é bimodal (um vale entre dois picos, figura 16).

2) *Limiarização pelo Método de Otsu*: o método de Otsu define um limiar global ótimo para ser usado na binarização da imagem. Ele é bom para ser aplicado em imagens com as mesmas circunstâncias ditas na limiarização global simples. O método consiste em dividir a imagem em duas classes: uma para os valores de *background* (fundo) e a outra para os valores de *foreground* (objeto). A partir disso, a probabilidade de um pixel pertencer a cada classe é calculada com base na frequência de ocorrência dos valores de intensidade no histograma. O método otimiza a busca de encontrar um limiar que maximize a variância entre as classes e minimize a variância dentro de cada classe.

3) *Limiarização Adaptativa pelo Método de Otsu*: para cada janela da imagem o método de Otsu é chamado para definir um limiar.

4) *Limiarização Adaptativa por Média*: para cada janela da imagem é calculado a média dos valores da janela e subtraído uma constante c para melhorar o resultado da limiarização. Um c com valor positivo reduz ruídos, enquanto um c com valor negativo aumenta a captura de detalhes do objeto [16].

5) *Limiarização Adaptativa por Bernsen*: para cada janela da imagem o limiar é calculado através da média aritmética do mínimo e do máximo valor entre os valores da janela [14].

F. Morfologia

De acordo com [8],

morfologia matemática consiste em extrair as informações relativas à geometria e à topologia de um conjunto desconhecido (uma imagem), pela transformação através de outro conjunto completamente definido, chamado elemento estruturante. Portanto, a base da morfologia matemática é a teoria de conjuntos.

Neste trabalho, as operações de morfologia são aplicadas em imagens binarizadas, portanto antes de receber uma operação morfológica a imagem receberá uma operação de segmentação. O uso do elemento estruturante na imagem é análogo ao uso de máscaras de convolução em filtragem espacial, mas com a principal diferença de que as operações morfológicas são não-lineares, e os valores dos pixels de saída

são gerados através de uma interação de cada pixel da matriz com um elemento estruturante. Em seguida, serão abordadas as técnicas de morfologia utilizadas nesta aplicação.

1) *Erosão*: diminui o *foreground* e aumenta o *background* da imagem, reduzindo o tamanho dos objetos de interesse. Na erosão é aplicado o conceito de *fit* do elemento estruturante à imagem. Para cada pixel da imagem, o elemento estruturante é movido de forma que seu pixel central esteja sobre o pixel correspondente da imagem. Caso todos os pixels 1 do elemento estruturante estiverem sobre os pixels 1 da imagem, considerando o tamanho da janela do elemento estruturante, o pixel da imagem será 1. Caso contrário, se algum pixel 1 do elemento estruturante não coincidir, o pixel da imagem recebe o valor 0.

2) *Dilatação*: diminui o *background* e aumenta o *foreground* da imagem, aumentando o tamanho dos objetos de interesse. Na dilatação é aplicado o conceito de *hit* do elemento estruturante à imagem. Para cada pixel da imagem, o elemento estruturante é movido de forma que seu pixel central esteja sobre o pixel correspondente da imagem. Caso algum pixel 1 do elemento estruturante estiver sobre os pixels 1 da imagem ou todos os pixels 1, considerando o tamanho da janela do elemento estruturante, o pixel da imagem será 1. Caso contrário, se nenhum pixel 1 coincidir, o pixel da imagem recebe o valor 0.

3) *Abertura*: aplica a operação de erosão seguida da operação de dilatação. A abertura é útil pois, suaviza o contorno de uma imagem e remove detalhes estreitos, como linhas finas, estejam elas conectando objetos ou não [8].

4) *Fechamento*: aplica a operação de dilatação seguida da operação de erosão. O fechamento é útil pois, fecha lacunas entre objetos, expande áreas vazias ou bordas do objeto, elimina pequenos buracos e preenche falhas no contorno [8].

III. SISTEMA PROPOSTO

O Photoshop da Julia é um programa de edição de imagens capaz de aplicar filtros passa-baixa, filtros passa-alta, operações de segmentação, erosão, dilatação, abertura ou fechamento. As edições são feitas utilizando uma versão da imagem que já teve um filtro ou operação aplicada anteriormente, exceto na primeira edição, em que a imagem original será usada. O Photoshop da Julia é uma aplicação utilizando a linguagem de programação Python 3.12 com as bibliotecas Pillow 10.4, OpenCV 4.10, NumPy 2.1 e CustomTkinter 5.2.2. Para melhor compreensão e organização deste artigo foi decidido a não inserção do código no documento. Porém, ele poderá ser acessado em https://github.com/juliarobaina/Photoshop_da_Julia/tree/etapa3.

A. Interface

O design da interface foi criado no Figma e pode ser acessado em <https://www.figma.com/design/oWGHsNoIvGfunt7DKEeUIw/Photoshop-da-Julia?node-id=26-97&t=96IXsIvdZHR6vM4p-1>. A cores da aplicação foram geradas utilizando o site Colors e o contraste dessas cores foram avaliadas com a ferramenta de acessibilidade do

264 Adobe Colors. Os ícones foram selecionados a partir da
 265 seção de ícones do Google Fonts. As inspirações para o
 266 desenvolvimento do design foram extraídas do site Dribbble.
 267 Na figura 17 é possível ver a imagem da tela principal.

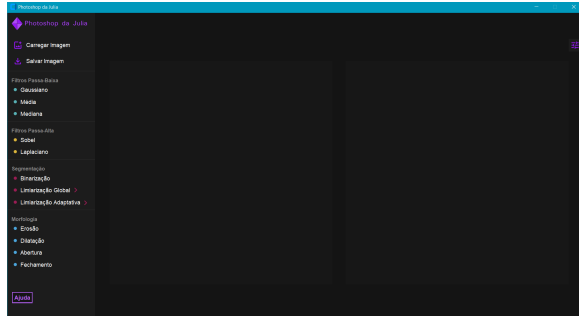


Fig. 17. Tela Principal do Photoshop da Julia

268 A seguir será apresentado uma descrição das funções relativas
 269 a todas as funcionalidades e funções auxiliares presentes no
 270 código do programa.

271 B. Funções Auxiliares para os Filtros Passa-Baixa e Passa- 272 Alta

273 1) *matrizComBordasZeradas*: retorna uma matriz contendo
 274 a matriz imagem adicionado de bordas, também chamado de
 275 *padding*s, com valor 0.

276 2) *matrizComBordasGemeas*: Retorna uma matriz com a
 277 matriz imagem adicionado de bordas com valores iguais as
 278 linhas e colunas da extremidade da matriz imagem.

279 3) *convolucao*: faz a operação de convolução entre uma
 280 tabela e um kernel. Essa função é utilizada para os filtros
 281 média, gaussiano, sobel e laplaciano.

282 4) *escalaDeCinza*: converte a imagem da escala BGR para
 283 a escala de cinza a partir de um cálculo de intensidade de luz
 284 em cada canal [17].

285 5) *mediana*: cria a matriz imagem com valores medianos,
 286 de acordo com o cálculo dito na subseção filtro mediana.

287 C. Filtros Passa-Baixa e Passa-Alta

288 1) *filtroMedia*: divide os canais B, G, R; para cada canal,
 289 invoca as funções *matrizComBordasZeradas* e *convolucao*; ao
 290 final, une os três canais e retorna a matriz imagem com a
 291 aplicação do filtro.

292 2) *filtroMediana*: divide os canais B, G, R; para cada canal,
 293 invoca as funções *matrizComBordasGemeas* e *convolucaoMe-*
 294 *diana*; ao final, une os três canais e retorna a matriz imagem
 295 com a aplicação do filtro.

296 3) *filtroSobel*: aplica um filtro gaussiano de kernel 3x3;
 297 converte a imagem de BGR para a escala de cinza; invoca
 298 as funções *matrizComBordasZeradas* e *convolucao*, uma vez
 299 para obter a matriz Gx e outra vez para a matriz Gy; soma Gx
 300 com Gy; ao final, retorna a matriz resultante do somatório.

301 4) *filtroLaplaciano*: aplica um filtro gaussiano de kernel
 302 3x3; converte a imagem de BGR para escala de cinza; invoca
 303 as funções *matrizComBordasZeradas* e *convolucao*; retorna a
 304 matriz convoluída.

305 5) *kernelMedia*: retorna uma matriz com tamanho 3x3, 5x5
 306 ou 7x7 para a operação de convolução do filtro média.

307 6) *kernelGaussiano*: retorna uma matriz com tamanho 3x3,
 308 5x5 ou 7x7 para a operação de convolução do filtro gaussiano.

309 7) *kernelSobel*: retorna as matrizes com tamanho 3x3 para
 310 a operação de convolução do filtro sobel.

311 8) *kernelLaplaciano*: retorna uma matriz com tamanho 3x3
 312 para a operação de convolução do filtro laplaciano.

313 D. Operações de Segmentação

314 1) *binarizacao*: recebe um valor de limiar para binarizar
 315 a imagem. Para a aplicação se o valor do pixel da matriz for
 316 maior que o limiar, então o pixel recebe o valor 0. Senão, o
 317 pixel recebe o valor 255. Terminado esse processo a matriz é
 318 retornada.

319 2) *limiarizacaoGlobal*: aplica a limiarização global simples
 320 usando a média dos valores da matriz com o limiar pré-
 321 determinado; retorna o valor de limiar global.

322 3) *metodoOtsu*: cria um histograma da imagem; normaliza
 323 os valores do histograma; aplica o método de Otsu. As
 324 equações de variância usadas na implementação podem ser
 vistas nas figuras 18 e 19; retorna a matriz limiarizada.

$$\sigma_W^2 = W_b \sigma_b^2 + W_f \sigma_f^2 \quad (16)$$

Fig. 18. Variância intra-classes Fonte: [18]

$$\sigma_b^2 = \sigma^2 - \sigma_W^2 \quad (17)$$

$$= W_b (\mu_b - \mu)^2 + W_f (\mu_f - \mu)^2 \quad (18)$$

$$\text{Where } \mu = W_b \mu_b + W_f \mu_f \quad (19)$$

$$W_b W_f (\mu_b - \mu)^2 \quad (20)$$

Fig. 19. Variância entre-classes Fonte: [18]

326 4) *limiarizacaoAdaptativaSemPaddingOtsu*: nesta função
 327 não há a adição de bordas na tabela porém o algoritmo percorre
 328 a matriz como se houvesse, a ideia pode ser visualizada na
 329 figura 20; aqui é aplicado o método de Otsu para obter um
 330 limiar; a função *binarizacao* é chamada para binarizar a parte
 331 que está sendo analisada da imagem (janela); retorna a matriz
 332 limiarizada.

333 5) *limiarizacaoAdaptativaMedia*: o algoritmo de
 334 limiarização adaptativa por média implementado leva
 335 em consideração diferentes tamanhos de janelas. A seguir,
 336 é descrito o processo de aplicação do ajuste do tamanho da
 337 janela nesse algoritmo:

338 1) o primeiro passo é verificar se o tamanho da janela
 339 (definido pelo usuário) é maior que o número de linhas
 340 ou colunas da matriz. Se for o caso, o algoritmo calcula
 341 o máximo divisor comum (MDC) entre o número de
 342 linhas e o número de colunas da matriz. Esse cálculo

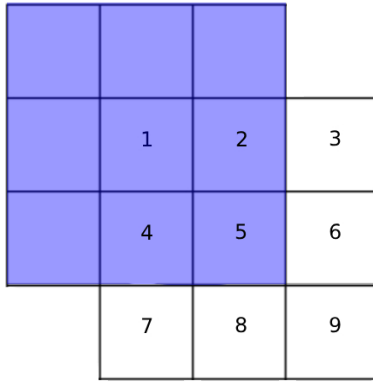


Fig. 20. Simulação da execução do algoritmo de limiarização adaptativa sem bordas com uma janela de 3x3

determina o tamanho ideal das submatrizes em que a matriz será dividida;

- 2) se o MDC calculado for maior que 1, a matriz será dividida em submatrizes com dimensões iguais ao MDC. Se o MDC for igual a 1, o tamanho da janela será mantido como a matriz inteira ou será aplicado para os valores restantes. Caso o tamanho da janela seja menor ou igual ao tamanho da matriz, a matriz será dividida no tamanho da janela especificado pelo usuário.

Para cada janela, o algoritmo calcula o limiar para a binarização. Com o limiar calculado, a função de binarização compara os valores dentro janela com o limiar. Após a aplicação da binarização em todas as janelas, o algoritmo retorna a matriz binarizada resultante.

6) *limiarizacaoAdaptativaBernsen*: aplica a mesma ideia que o algoritmo de limiarização adaptativa média com a diferença de que o cálculo da média é feito como está descrito no item 5 da subseção segmentação; matriz modificada é retornada.

E. Operações de Morfologia

O elemento estruturante usado durante a execução dos algoritmos de morfologia é uma matriz com todos os seus valores iguais a 1, um exemplo pode ser visto na figura 21.

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (21)$$

Fig. 21. Elemento Estruturante 3x3

1) *erosao*: a função *matrizComBordasZeradas* é chamada para adicionar bordas à matriz imagem; um *loop* é executado para verificar as correspondências do tipo *fit* do elemento estruturante com a tabela; a matriz é retornada.

2) *dilatacao*: a função *matrizComBordasZeradas* é chamada para adicionar bordas à matriz imagem; um *loop* é executado para verificar as correspondências do tipo *hit* do elemento estruturante com a tabela; a matriz é retornada.

3) *abertura*: é chamado a função de erosão e depois dilatação; matriz é retornada.

4) *fechamento*: é chamado a função de dilatação e depois erosão; matriz é retornada.

IV. RESULTADOS

As imagens mencionadas nesta seção poderão ser visualizadas na seção VI.

A. Filtros Passa-Baixa

Nesta etapa foram utilizadas imagens com ruído sal e pimenta e gaussiano. Ruído sal e pimenta, "contém apenas dois valores, um próximo ao valor máximo (sal) e o outro próximo ao valor mínimo (pimenta) da escala de cinza" [8]. Ruído gaussiano, a distribuição dos valores da escala de cinza da imagem possuem a forma gaussiana [8].

Ao realizar o teste de imagens com esses ruídos, o que foi dito em relação ao filtro de mediana e média na subseção Filtro Passa-Baixa, de que o filtro mediana tem um desempenho melhor no ruído sal e pimenta do que no ruído gaussiano comparado ao filtro de média pôde ser observado. Também é perceptível que o borrimento na imagem com o filtro gaussiano do ruído gaussiano é mais suave do que os demais e as linhas de expressão do bebê e contornos são mais preservados. Além disso, com o ruído sal e pimenta o filtro gaussiano foi a imagem em que os ruídos ficaram mais vivos. Esses apontamentos podem ser observados da figura 22 até a 29.

B. Filtros Passa-Alta

Foi observado que as amostras da figura 30 tem uma quantidade maior de bordas únicas, mais evidentes e com mais ruído no filtro sobel. Com o filtro laplaciano linhas na imagem criadas pela incidência da luz também foram identificadas. Para a figura 33 o filtro laplaciano destaca as linhas de contorno do gato, já o sobel destaca quase todo o animal.

Nota-se que nas duas imagens testadas é possível verificar, que apesar do ruído, as características de cada filtro podem ser observadas. Filtro sobel na figura 30 e filtro laplaciano na figura 33.

C. Segmentação

Para algumas imagens, foi realizado um estudo comparativo entre as funções implementadas neste trabalho e as funções presentes na biblioteca OpenCV, isso porque a limiarização global simples e a limiarização de Bernsen não existem no OpenCV. A seguir estão os resultados.

1) *Binarização*: Não houve diferenças entre as imagens, com a função *binarizacao* e a função *cv2.threshold* com o parâmetro *THRESH_BINARY_INV*, figura 38 e figura 37. O valor do limiar utilizado foi igual a 3.

2) *Método de Otsu Global*: A função *cv2.threshold* com o parâmetro *THRESH_BINARY_INV* e *THRESH_OTSU* apresentou uma presença ínfima de ruídos comparado a função *metodoOtsu*, figura 40 e figura 41.

3) *Limiarização Global Simples*: apresentou melhor desempenho para imagens com pouca ou nenhuma variação de iluminação e poucas informações. Essa observação está nas figuras 42 e 44. O limiar inicial definido é igual a 3.

4) *Limiar Adaptativo de Otsu*: foi usado a função `cv2.threshold` com o parâmetro `THRESH_BINARY_INV` e `THRESH_OTSU` na função `limiarizacaoAdaptativaOtsu` substituindo o método criado para este trabalho. Não aconteceu alterações no uso da função da biblioteca OpenCV com a função do editor de imagens, figura 45 e figura 46. O tamanho da janela usada é igual a 77.

5) *Limiarização Adaptativa de Bernsen*: na figura 47 é possível perceber que o método conseguiu dar um bom destaque para todos os números presentes no jogo. O tamanho da janela foi 103.

6) *Limiarização Adaptativa por Média*: na figura 48 e 49 é visível que a função média descrito no trabalho está com mais ruído do que a função `cv2.adaptiveThreshold` com o parâmetro `cv2.ADAPTIVE_THRESH_MEAN_C`. O tamanho da janela foi 15 com a constante c igual a 6.

D. Morfologia

Para todas as imagens, foi realizado um estudo comparativo entre as funções implementadas neste trabalho e as funções presentes na biblioteca OpenCV. A seguir estão os resultados.

1) *Erosão*: nas figuras 50 e 51 é exibido a erosão de duas imagens segmentadas com o método de Otsu. Tanto com a função `cv2.erode` quanto com a função `erosao`, é possível observar que não há diferenças entre as duas imagens.

2) *Dilatação*: nas figuras 52 e 53 é exibido a dilatação de duas imagens segmentadas com a limiarização por média. Tanto com a função `cv2.dilate` quanto com a função `dilatacao`, é possível observar que há mínimas diferenças entre as duas imagens.

3) *Abertura*: nas figuras 54 e 55 é exibido a abertura de duas imagens segmentadas com o método de otsu. Tanto com a função `cv2.erode` seguida da função `cv2.dilate` quanto com a função `abertura`, é possível observar que não há diferenças entre as duas imagens.

4) *Fechamento*: nas figuras 56 e 57 é exibido a abertura de duas imagens segmentadas com o método de Otsu. Tanto com a função `cv2.dilate` seguida da função `cv2.erode` quanto com a função `fechamento`, é possível observar que não há diferenças entre as duas imagens.

V. CONCLUSÃO

Foram mostrados no texto, os conceitos de filtros no domínio do espaço, segmentação e morfologia e operações relacionadas implementadas na aplicação Photoshop da Julia. No entanto, para imagens com muita informação o tempo de execução da aplicação das funções com sua saída pode ser demorado, ultrapassando 3 minutos de espera. Fica evidente, portanto, que todos os filtros, operações de segmentação e morfologia explicados são executados no editor e o objetivo de aplicar conhecimento teórico da área de processamento de imagens na prática através do desenvolvimento de um editor de imagens foi alcançado.



Fig. 22. Imagem com ruído sal e pimenta Fonte: [8]



Fig. 23. Imagem com ruído sal e pimenta aplicado o filtro média com kernel 5x5



Fig. 24. Imagem com ruído sal e pimenta aplicado o filtro mediana com kernel 5x5



Fig. 26. Imagem com ruído gaussiano Fonte: [8]



Fig. 25. Imagem com ruído sal e pimenta aplicado o filtro gaussiano com kernel 5x5



Fig. 27. Imagem com ruído gaussiano aplicado o filtro média com kernel 5x5

REFERÊNCIAS

- [1] I. N. de Pesquisas Espaciais INPE. (2006) Processamento de imagens. [Online]. Available: https://www.dpi.inpe.br/spring/portugues/tutorial/introducao_pro.html
- [2] IllidanStormrage. (2022) Operador sobel. [Online]. Available: <https://www.cnblogs.com/IllidanStormrage/p/16206945.html>
- [3] C. H. Sanches, P. J. Fontoura, P. F. Viera, and M. A. Batista, "Técnicas de suavização de imagens e eliminação de ruídos," in *Anais da VI edição do Encontro Anual de Tecnologia da Informação*. Santa Maria, RS: Instituto Federal Farroupilha Campus de Frederico WestphalenUniversidade Federal de Santa Maria and Universidade Federal de Santa Maria - Campus Frederico Westphalen, 2015, pp. 21–30. [Online]. Available: <http://anais.eati.info:8080/index.php/2019/article/view/146>
- [4] A. S. Catarina, "Filtragem espacial," <https://www.inf.unioeste.br/~adair/PID/Notas%20Aula/Processamento%20de%20Imagens%20Digitais%20-%20Filtragem%20Espacial.pdf>, 2023.
- [5] S. Cáceres, "Filtragem no domínio espacial," <https://sheilacaceres.com/courses/tapfe/pi/PI-7-Filtragem.pdf>, 2015.
- [6] B. J. Dembogurski, "Filtragem no domínio espacial," Moodle++ DCC, 2024.
- [7] C. D. Q. OLIVEIRA, "Metodologia de processamento digital aplicada à exploração de imagens radiológicas," Mestrado em Engenharia Nuclear, Instituto Militar de Engenharia, Rio de Janeiro, 2004, orientadores: Prof. Julio Estrada and Prof. Rubemar S. Ferreira. [Online]. Available: <https://inis.iaea.org/collection/NCLCollectionStore/>



Fig. 28. Imagem com ruído gaussiano aplicado o filtro mediana com kernel 5x5



Fig. 29. Imagem com ruído gaussiano aplicado o filtro gaussiano com kernel 5x5

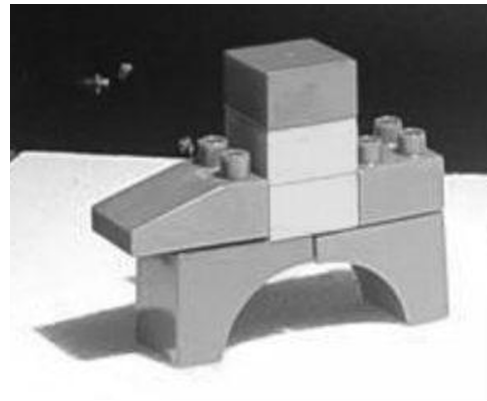


Fig. 30. Imagem Original Fonte: [6]

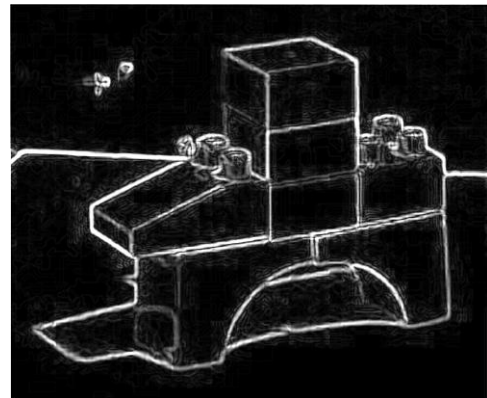


Fig. 31. Imagem com filtro sobel com máscaras 3x3

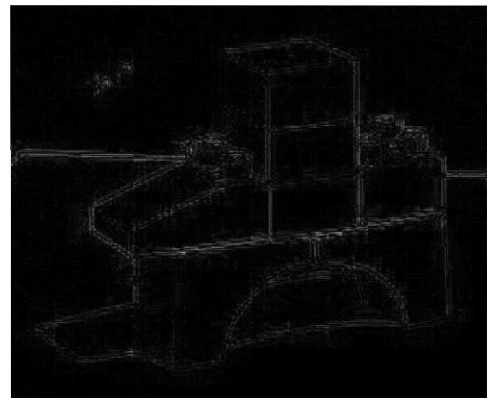


Fig. 32. Imagem com filtro laplaciano com kernel 3x3

508 _Public/40/040/40040518.pdf
 509 [8] O. M. Filho and H. V. Neto, *Processamento de Imagens*. Rio de Janeiro,
 510 Rio de Janeiro: Editora Brasport, 1999.
 511 [9] L. L. de Oliveira Junior, "Filtros compostos e adaptativos: o filtro de
 512 gaussiano, laplaciano do gaussiano e de gabor (harmônico-gaussiano)," <http://profs.ic.uff.br/~aconci/gabor.pdf>, 2024.
 513 [10] A. Huamán. (2024) Sobel derivatives. [Online]. Available: https://docs.opencv.org/4.x/d2/d2c/tutorial_sobel_derivatives.html
 514 [11] L. B. T. Gadioli, V. L. G. da Silva, and L. A. Nascimento,
 515 "Comparação de métodos de detecção de bordas em imagens
 516 de satélite de florestas de eucalipto no vale do jequitinhonha,"
 517 in *Anais do XIX Simpósio Brasileiro de Sensoriamento*
 518 *Remoto*. Santos, SP: INPE São José dos Campos, 2019.
 519 [Online]. Available: <https://proceedings.science/sbsr-2019/trabalhos/>
 520
 521

comparacao-de-metodos-de-deteccao-de-bordas-em-imagens-de-satelite-de-florestas?
 lang=pt-br 523
 [12] OpenAI, "Chatgpt: Conversational ai," 2024. [Online]. Available:
 524 <https://www.openai.com/chatgpt> 525
 [13] V. Mecânica. (2024) What are the advantages and 526
 527 disadvantages of using sobel, canny, and laplacian filters for
 528 edge detection? [Online]. Available: [https://www.linkedin.com/](https://www.linkedin.com/advice/1/what-advantages-disadvantages-using-sobel-canny?lang=en&originalSubdomain=pt)
 529 [advice/1/what-advantages-disadvantages-using-sobel-canny?lang=en&](https://www.linkedin.com/advice/1/what-advantages-disadvantages-using-sobel-canny?lang=en&originalSubdomain=pt)
 530 [originalSubdomain=pt](https://www.linkedin.com/advice/1/what-advantages-disadvantages-using-sobel-canny?lang=en&originalSubdomain=pt) 531
 [14] S. M. Victor and B. S. Silva, "Estudo comparativo de técnicas de
 532 limiarização de histogramas em imagens de nódulos de mamografia,"
 533 in *XXI ENMC (Encontro Nacional de Modelagem Computacional)*



Fig. 33. Imagem Original Fonte: [19]



Fig. 34. Imagem com filtro sobel com máscaras 3x3



Fig. 35. Imagem com filtro laplaciano com kernel 3x3



Fig. 36. Imagem Original Fonte: [20]



Fig. 37. Imagem com a função binarizacao



Fig. 38. Imagem com a função cv2.threshold com o parâmetro THRESH_BINARY_INV

- 01. Paranaguá, PR: Revista Mundi Engenharia, Tecnologia e 535
Gestão, 2019. [Online]. Available: [https://revistas.ifpr.edu.br/index.php/](https://revistas.ifpr.edu.br/index.php/mundietg/article/view/1090) 536
[mundietg/article/view/1090](https://revistas.ifpr.edu.br/index.php/mundietg/article/view/1090) 537

[15] S. Cáceres, "Histograma da imagem," [https://sheilacaceres.com/courses/](https://sheilacaceres.com/courses/pi/aula4/PI-4-Histograma.pdf) 538
[pi/aula4/PI-4-Histograma.pdf](https://sheilacaceres.com/courses/pi/aula4/PI-4-Histograma.pdf), 2015. 539

[16] A. Yadav. (2024) Opencv adaptive threshold. [Online]. Available: <https://> 540



Fig. 39. Imagem Original Fonte: [21]



Fig. 41. Imagem com a função `cv2.threshold` com o parâmetro `THRESH_BINARY_INV` com `THRESH_OTSU`



Fig. 40. Imagem com a função `metodoOtsu`



Fig. 42. Imagem com a função `limiarGlobalSimples`

- 541 //medium.com/@amit25173/opencv-adaptive-threshold-fae667b91984
 542 [17] Wikipédia. (2020) Nível de cinza. [Online]. Available: [https://pt.](https://pt.wikipedia.org/wiki/N%C3%ADvel_de_cinza)
 543 [wikipedia.org/wiki/N%C3%ADvel_de_cinza](https://pt.wikipedia.org/wiki/N%C3%ADvel_de_cinza)
 544 [18] M. Razali, "Segmentation process part 2: Otsu explanation and
 545 calculation (vision system) -w5;" 2020. [Online]. Available: [https:](https://www.youtube.com/watch?v=0DFdT0OWLeo&t=1250s)
 546 [//www.youtube.com/watch?v=0DFdT0OWLeo&t=1250s](https://www.youtube.com/watch?v=0DFdT0OWLeo&t=1250s)
 547 [19] Imagem, "Imagem," Não Disponível.
 548 [20] B. J. Dembogurski, "Processamento morfológico de imagens," Moo-
 549 dle++ DCC, 2024.
 550 [21] —, "SegmentaÇão de imagens," Moodle++ DCC, 2024.
 551 [22] OpenCV. (2024) Image thresholding. [Online]. Available: [https:](https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html)
 552 [//docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html](https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html)



Fig. 43. Imagem Original Fonte: [22]

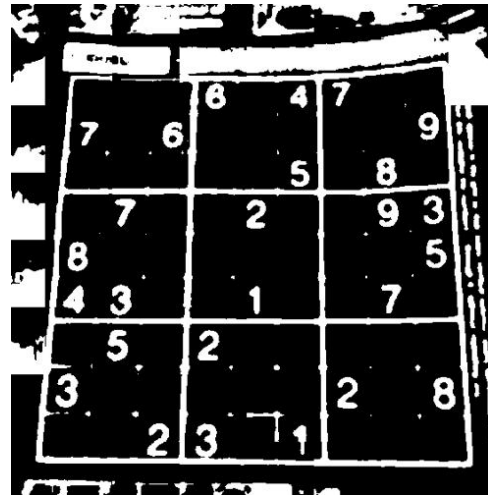


Fig. 46. Imagem com a função A função cv2.threshold com o parâmetro THRESH_BINARY_INV e THRESH_OTSU usada na função limiarizacaoAdaptativaOtsu



Fig. 44. Imagem com a função cv2.threshold com o parâmetro THRESH_BINARY_INV e THRESH_OTSU

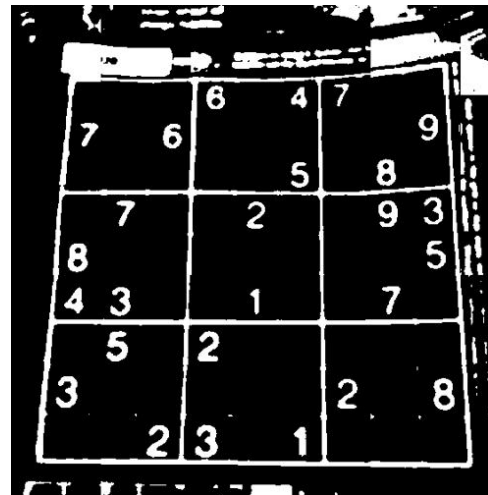


Fig. 47. Imagem com a função limiarizacaoBernsen

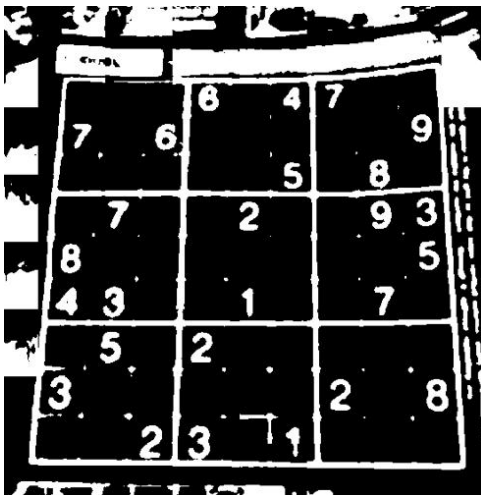


Fig. 45. Imagem com a função limiarizacaoOtsu

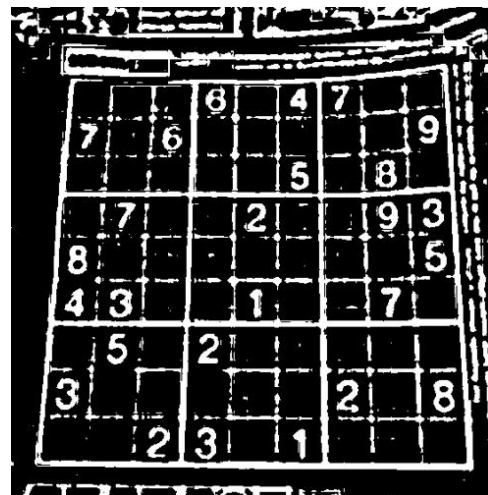


Fig. 48. Imagem com a função limiarizacaoAdaptativaMedia

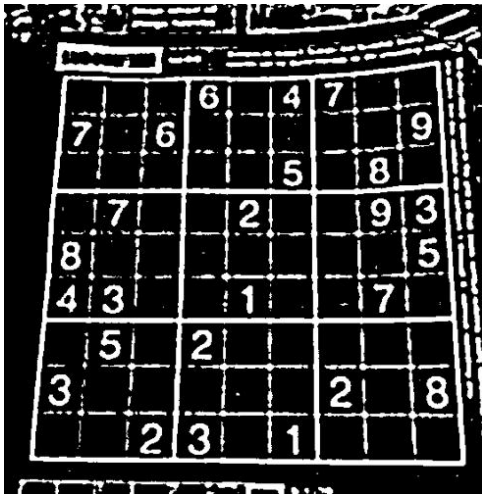


Fig. 49. Imagem com a função `cv2.adaptiveThreshold` com o parâmetro `cv2.ADAPTIVE_THRESH_MEAN_C`



Fig. 52. Imagem com a função dilatação



Fig. 50. Imagem com a função erosão



Fig. 53. Imagem com a função `cv2.dilate`



Fig. 51. Imagem com a função `cv2.erode`

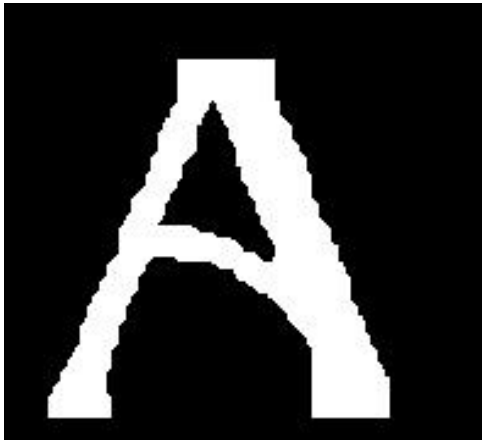


Fig. 54. Imagem com a função abertura



Fig. 55. Imagem com a função `cv2.erode` seguida de `cv2.dilate`



Fig. 57. Imagem com a função `cv2.dilate` seguida de `cv2.erode`



Fig. 56. Imagem com a função fechamento